Berlin University of Applied Sciences

INTELL-E-READER

WHERE READING MEETS TECHNOLOGY

PROJECT BY:

Prajyot Suvarna

&

Sujith Gunjur Umapathy

MAJORS IN

MSc. Big Data

&

Artificial Intelligence

Guided By:

Prof: Ganesh Tottempudi

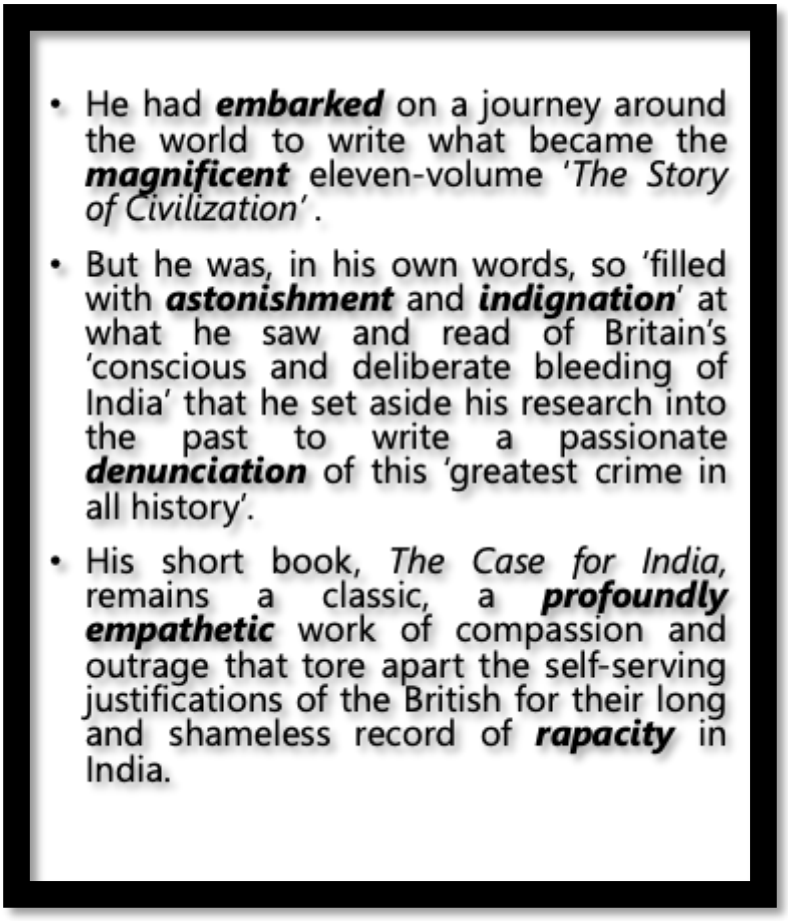
Table of Contents

1. INTRODUCTION

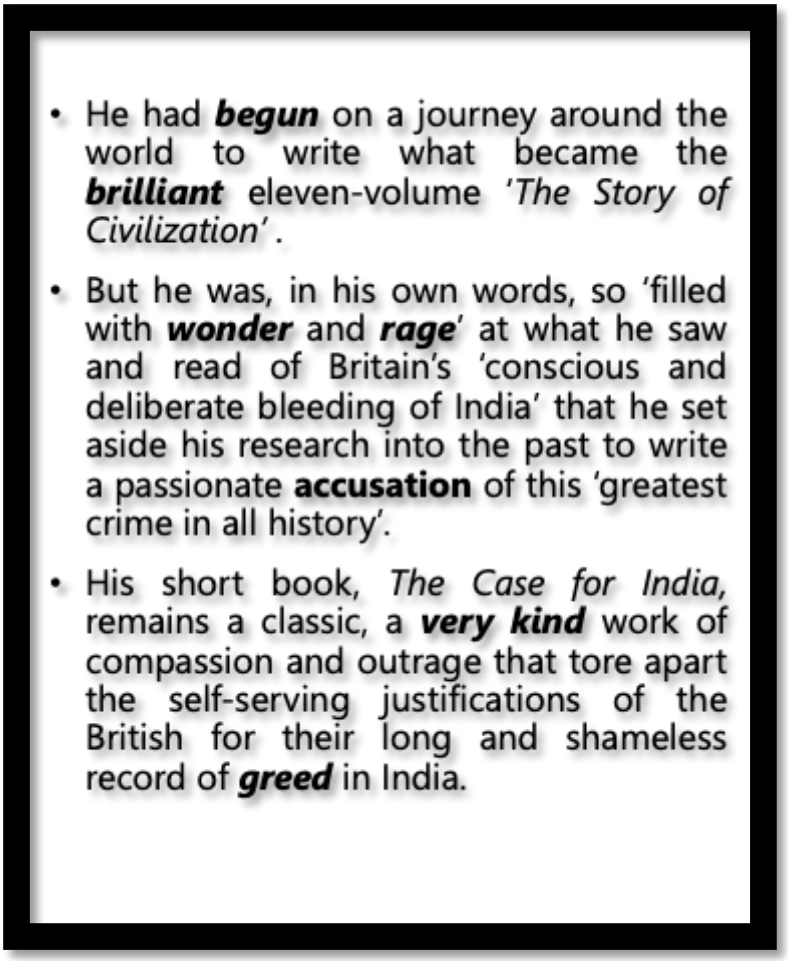
How often does it happen that you read a book in a language you're getting acquainted with? Apart from reading books for the story, many of us also read books in order to enhance our vocabulary. A survey conducted gave us the results that many people find it unsatisfactory to constantly look up the meaning of words while reading a book which takes away from their immersion in the story and thus subsequently their interest in the book.

Intell-e-Reader aims to solve this problem. Our solution is based on the Common European Framework (CEFR) system of categorizing words into 3 primary difficulties A (A1 and A2), B (B1 and B2) and C (C1 and C2) and then replacing the words with the higher difficulty levels (B and C) with their simpler synonyms.

Let us take some sentences from Shashi Tharoor's book for our reference.



Here the words which would seem difficult to a novice English learner and highlighted in bold. Our aim is to reduce the complexity of these words by finding their simpler synonyms. An example of the modified text would be as follows.



From this it's quite evident that the reader has to look up their dictionaries way less often as resolving the complicated words with its simpler alternative provides for an easier and smoother reading experience.

2. WORKFLOW

2.1 Building a Data Set

Our project is primarily driven by the Common European Framework (CEFR) standard. The Common European Framework of Reference for Languages (CEFR) is an international standard for describing language ability. It describes language ability on a six-point scale, from A1 for beginners, up to C2 for those who have mastered a language. This makes it easy for anyone involved in language teaching and testing, such as teachers or learners, to see the level of different qualifications [2.1].

This framework lays a strong ground for us to understand words which are easily understandable for a beginner in English language and pose a challenge to the aspirants who are in seek of new and comprehensive English words which are categorised under C1 and C2 scale of the CEFR standard.

Our first challenge henceforth to achieve a solution to our problem statement is to identify a data set which can give us information about the English word and its corresponding CEFR level. The first source of information that helped us proceed forward was a research dataset that was collected from a group of 3 English teachers [2.2]. This data set comprises of 7000 English words along with their Part of Speech (PoS) and the CEFR level derived by the 4 methods. Namely:

1. Averaging the CEFR scores rated by 3 Teachers for the same word.
2. Random Forest – Derived by training the model with the Teachers Average along with Google Books 1-Gram Data.
3. Neural Network– Derived by training the model with the Teachers Average along with Google Books 1-Gram Data.
4. Standard Vector Machine– Derived by training the model with the Teachers Average along with Google Books 1-Gram Data.

Data generated has the below structure:

Word	Part of Speech	CEFR Level derived by Teachers Average	CEFR Level derived by Random Forest	CEFR Level derived by Neural Network	CEFR Level derived by Standard Vector Machine

Once the above data is generated, we calculate the average of the 4 different CEFR levels and arrive at a CEFR level. Furthermore, the 6 standard point scale of the CEFR framework is reduced to 3 levels for the sake of algorithmic simplicity to – A, B & C level.

A – Comprises of all the words falling under Level A1 & A2
B – Comprises of all the words falling under Level B1 & B2
C – Comprises of all the words falling under Level C1 & C2

CEFR Level	CEFR Integer Level
A1	1
A2	2
B1	3
B2	4
C1	5
C2	6

[CEFR on a standard 6 Point Scale Vs Integer Value assigned for Algorithmic simplicity]

Reduced CSV file comprises of the following columns.

Word	Part of Speech	Average CEFR Level Integer Value	Average CEFR Level
Acquire	NOUN	4	B
Cash	NOUN	3	B

[Example for 2 words with their respective column data]

This dataset comprised of just 7000 words from the English language which was insufficient for us to produce the expected output. To enhance this data set, we decided to scrape the web page generated by English Profile® [2.3], which comprises of the English words along their PoS and CEFR Level. It is noteworthy to understand that this webpage emphasizes on the usage of standard English language norms to score their English words under a 6-point CEFR scale. Under standard British English language, we were able to get data of 15,696 words.

Columns scraped from the English Profile® webpage were:

Base Word	Level	Part of Speech

The new data set that we had scraped was later combined with the Teachers Average Data set. Further on by removing duplicates, words with no PoS and determinants (DET), our master CEFR data set comprised of 7621 words. This data set would further be used in identifying the replacement for words which are classified as complex [Complex words - Words with CEFR level >2 from our master data set, or words with CEFR Level B or C].

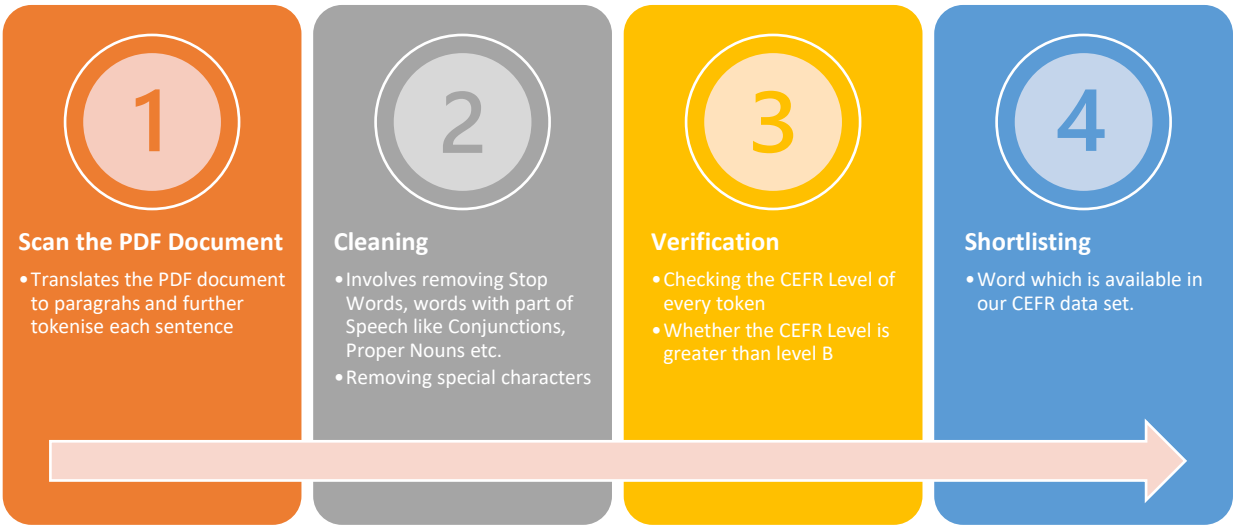
word	pos	cefr_int	cefr
caution	VERB	6	C
advertisement	NOUN	2	A
assert	VERB	5	C
address	VERB	6	C
cash	NOUN	3	B
acquire	VERB	4	B
click	VERB	6	C
bike	NOUN	3	B
clarification	NOUN	5	C
acknowledge	VERB	5	C
aid	VERB	5	C
applaud	VERB	6	C
cease	VERB	4	B

[Snippet from the file – master_cefr.csv]

2.2 IDENTIFY COMPLEX WORDS

Now that we hold a data bank which can inform us about a word's complexity, it is essential for us to identify the complex words in a PDF document that we upload every time. Here, the PDF document serves a document which requires us to operate upon and reduce the complex words that we have identified to a much simpler and easily understandable words.

This task was achieved based on the following algorithmic steps.



A final list is generated. It comprises of the Sentence, Words from that sentence and PoS of those Word.

1. Scan the PDF document: Using an online library PyMuPdf [2.4], we extract the information in the PDF file as paragraphs. These paragraphs are further broken down to sentences for simplification.
2. Cleaning: This phase operates on every sentence that has been extracted from the PDF file. Here we tokenise each sentence, followed by removing any special characters like [,.!:;] for comparison's sake. Tokens which are classified as Stop words, Sub-ordinating Conjunctions and Co-ordinating Conjunctions are rejected by our algorithm. In this process, the tokens are tagged with their parts of speech.
3. Verification: We verify whether the final list of words are available in our 'master_cefr' dataset. And if available, we do check whether the CEFR level of that word is either a B or C. This is because CEFR level A words are simple words and do not require further simplification.
4. Shortlisting: This process involves using only those words that have passed the verification stage and now are eligible to be further simplified.

The list structure for eligible words in the sentence are as follows:

{

[

sentence: // Sentence which was examined from a paragraph.

E.g., `What you mought call me? You mought call me captain`

refined_pos_tag_tuple: // consists of a list of words in the above sentence which pass the verification phase, along with their Part of Speech.

E.g., `[('mought', 'n'), ('call', 'n'), ('mought', 'v'), ('call', 'n'), ('captain', 'n')]`

period_break: // Indicates if the sentence consists of a period. (This is later used in regenerating the paragraphs)

E.g., `True` Since this life consists of a full stop.

]

}

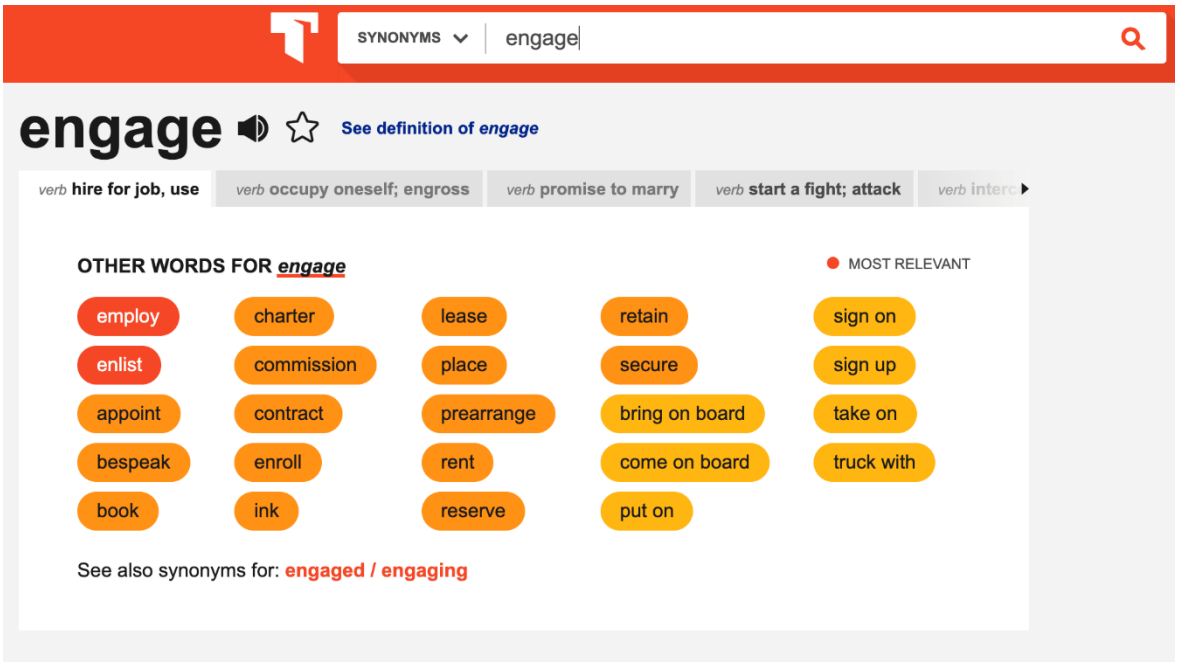
This list is referred to as the '*master_data*' in our code snippet. This list comprises of every eligible sentence from each paragraph along with their counterpart meta data. Upon identification of the words which we need to reduce the complexity, it is in the next step that we identify the list of synonym words and their CEFR level to shortlist the candidates which are potential replacements for a complex word.

2.3 FIND THE SYNONYMS

2.3.1 CHOOSING THE RIGHT WEBSITE

For finding the synonyms of a word, we identified web-scraping as the appropriate solution also enabling us to learn the process. Using python packages like 'BeautifulSoup' [2.3.1] and 'Requests' [2.3.2], we were able to ping a HTTP request to the website and extract the contents of its webpage.
There are many websites that allow the users to scrape information from them like merriam-webster.com or collinsdictionary.com but the following were shortlisted.

1. **Thesaurus** (www.thesaurus.com)

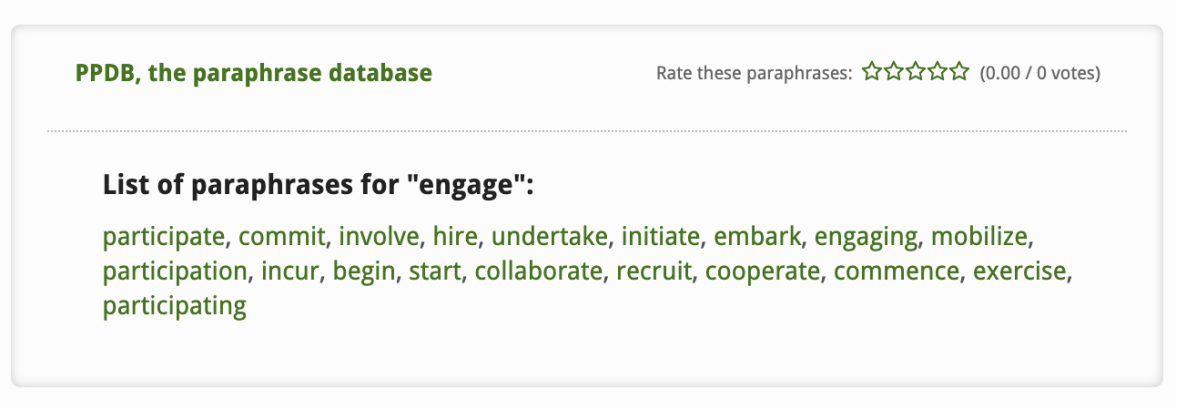


Advantages:
Thesaurus is a very powerful website as it segregates synonyms of the word based on their meanings and parts of speech. As some words can be classified as nouns as well as verbs.

One of the major advantages of this website is that it also categorizes the synonyms based on relevance. As you can see, red signifies the most relevant synonym while orange is moderately relevant and yellow is the least relevant synonym for the search word which is perfect for our requirements allowing us to pick only the most relevant synonyms if needed.

Limitation:
Although Thesaurus is perfect for our requirement, accessing data of different tabs means the use of chrome driver (**selenium**) along with BeautifulSoup which significantly increases the response time thus making it less than ideal.

2. Synonyms (www.synonyms.com)

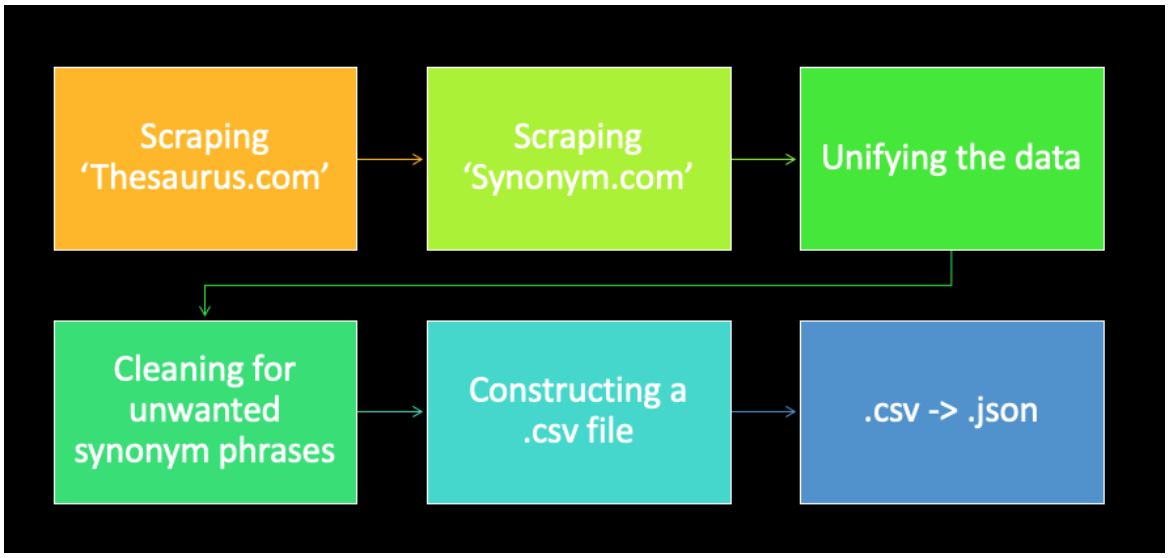


Advantages:
The biggest advantage of this site is that it has a section called “List of paraphrases” which gives the most relevant synonyms/paraphrases collating different meanings of the words and thus saving us the burden of using selenium.

Limitation:
Web-scraping this site takes longer time compared to Thesaurus.

2.3.2 PROCESS FLOW

This illustrates the basic flow of logic for fetching the synonyms:



2.3.3 SCRAPING APPROACH

For scraping of the synonyms, we could have followed any of the two approaches:

1. Static Scraping
2. Dynamic Scraping

Static Scraping: In this approach, we would need to scrape the synonyms ahead of time for the dataset we had.

Dynamic Scraping: In this approach, we would scrape the web as and when we would encounter a complex word in a sentence during processing.

The performance of the web-scraping on the sites compares as follows:

Website	Time taken for a word (approx)
www.thesaurus.com (single tab)	0.2s
www.thesaurus.com (multiple tabs)	20+s (depending on the number of tabs)
www.synonyms.com	3.2s

Fetching the synonyms dynamically would significantly increase the processing time for a page in a book. Assuming we find 30 complex words in a page, fetching synonyms for those words from www.thesaurus.com would take 6s whereas fetching those from www.synonyms.com would take about 96s which would make the application seem awfully slow and making user wait more than 4s for a response isn't ideal as it makes it seem that the application has crashed.

Keeping these observations in mind, we decided to follow the first approach. By using the static approach, we were able to run a script to fetch synonyms for the static data of around 8600+ words with complexities of B and above from www.thesaurus.com in about 4 hours whereas scraping words from www.synonyms.com took about 7.5 hours as they were synchronous requests made to the website. Then the data was stored locally in files.

2.3.4 Storing data in CSV format.

The next step in the process was storing the data locally in a format which can be easily accessed and manipulated. Storing the data in a .csv format provided us with both of those benefits. The synonyms fetched from both the sites were stored in separate .csv files.

Word	Synonyms
absolute_aj	['complete', 'full', 'infinite', 'outright', 'pure', 'sheer', 'simple', 'unadulterated', 'unconditional', 'unlimited', 'unqualified', 'utter']
abstract_aj	['abstruse', 'hypothetical', 'philosophical', 'unreal']
absurd_aj	['crazy', 'foolish', 'goofy', 'illogical', 'irrational', 'laughable', 'ludicrous', 'nonsensical', 'preposterous', 'silly', 'stupid', 'unreasonable', 'wacky']
academic_aj	['collegiate', 'intellectual', 'scholarly', 'scholastic']
acceptable_aj	['adequate', 'common', 'decent', 'fair', 'respectable', 'sufficient', 'tolerable']

SynListThesaurus.csv

For data extracted from Thesaurus.com, the key comprised of "word_partOfSpeech" due to it having their data in tabs for different parts of speech.

Word	Synonyms
absolute	['abject', 'utter', 'absolutely', 'top', 'complete', 'extreme', 'outright', 'mutlaq', 'overriding', 'total', 'full', 'unconditional', 'unqualified', 'definitely', 'laun
abstract	['summary', 'abstracto', 'abbreviated', 'summarized', 'abridged', 'abstracts', 'short', 'memorandum', 'summaries', 'résumé', 'synopsis', 'compendium'
absurd	['preposterous', 'ridiculous', 'nonsensical', 'nonsense', 'ludicrous', 'senseless', 'unreasonable', 'pointless', 'absurdly', 'perverse', 'irrational', 'aberrant'
academic	['university', 'academy', 'academics', 'universitaire', 'academia', 'scholarly', 'school', 'scholastic', 'theoretical', 'scholars', 'scolaire', 'universities', 'univ
acceptable	['unacceptable', 'accepted', 'tolerable', 'admissible', 'agreeable', 'permissible', 'palatable', 'satisfactory', 'reasonable', 'maqbool', 'decent', 'eligible', 'a
accessible	['available', 'access', 'affordable', 'accessibility', 'accessed', 'reachable', 'approachable', 'open', 'retrievable', 'attainable']

SynListSynonyms.csv

The data extracted from Synonyms.com was stored in key and value format where the key is the word, and the value is its synonyms.

2.3.5 Cleaning CSV data

The next step involved cleaning the data.
The following steps were done as part of data cleaning process:

1. Remove words with no synonyms.
2. Remove words containing phrases (with dashes) as synonyms.
3. Remove words with special characters in it.
4. Combine both the files with unique words.

2.3.6 Convert CSV to JSON

For the last step, the .csv files were converted to .json files and stored locally to have quick access to the synonym data by just calling the dict.get() function.

Also, different functions were written to either get synonyms based on the part of speech or to fetch all the synonyms from both the files.

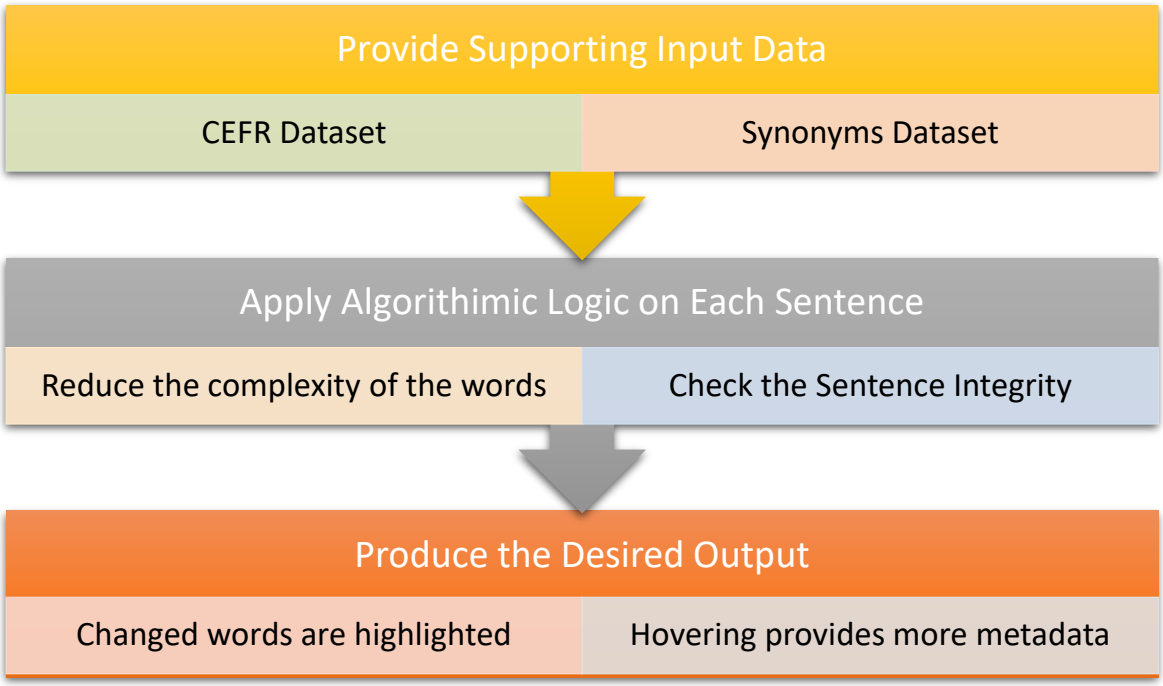
Convert CSV to JSON

In [ ]:
```
import json
import csv
from pathlib import Path

path = Path(__file__).parent.parent

# Converts 2 column CSV to JSON.
def convertToJsonFile(relPath, oldFileName, newFileName):
    try:
        # Read csv file.
        with open(str(path) + relPath + oldFileName) as csv_file:
            csv_reader = csv.reader(csv_file, delimiter=',')
            line_count = 0
            wordSynDict = {}
            for row in csv_reader:
                if line_count > 1:
                    rowObj = {
                        # key is the word and value is the synonyms as a list.
                        row[0]: row[1][1:-1].replace('\'','').replace(' ', '').split(',')
                    }
                    wordSynDict.update(rowObj)
                line_count += 1

        # Write to a new file.
        try:
            with open(str(path) + relPath + newFileName, 'w') as outfile:
                json.dump(wordSynDict, outfile)
            print('Success: File created')
        except:
            print('Error in creating file')

    except:
        raise Exception('File does not exist')
```

Code Snippet for converting csv to json.

2.4 CHECK SENTENCE INTEGRITY

Now that we have prepared the Synonyms dataset for the list of words available in the 'master_cefr' dataset, we can easily proceed with designing the algorithm to solve our problem statement. Overview on the problem-solving approach is discussed below. This approach is designed for a sentence. Combined effect on every sentence in the PDF provides the intended solution for the whole page.



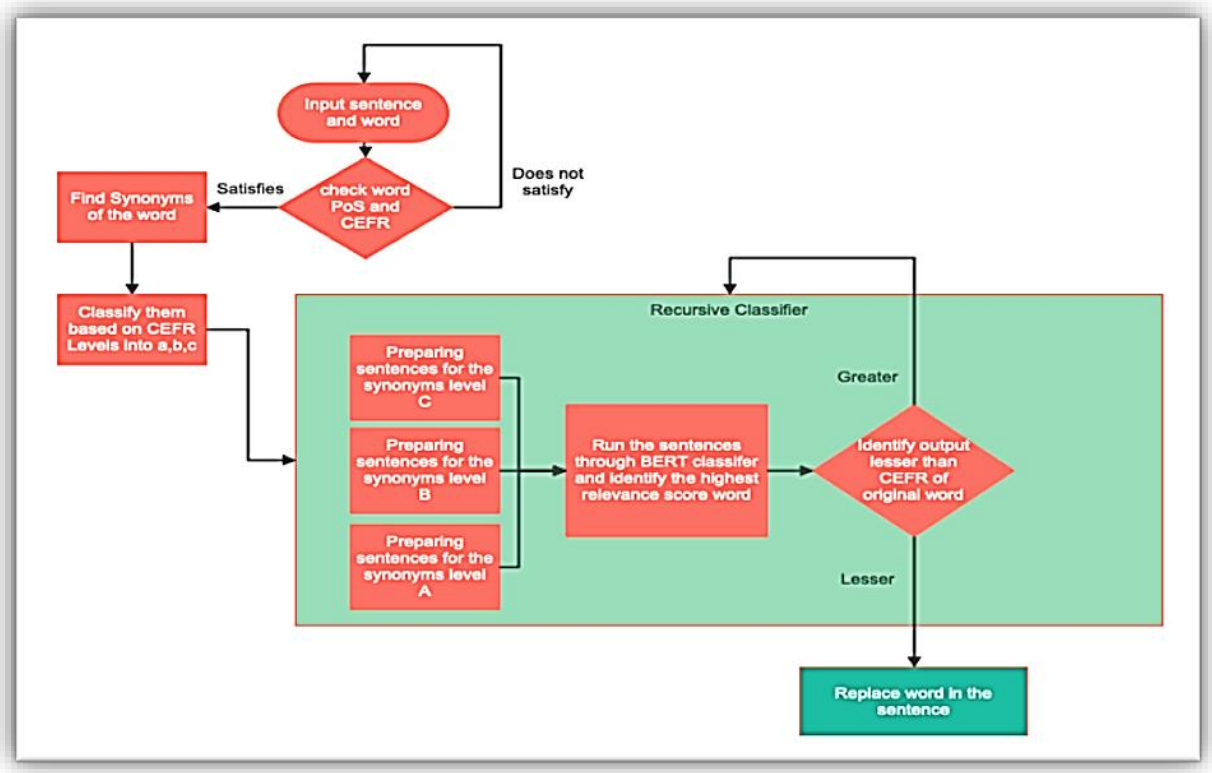
2.4.1 Provide Supporting Input Data

From the CEFR dataset and Synonyms data set we create the supporting input data to reduce the complexity of each sentence.

1. Under the '*Cefr'* class file, we retrieve the CEFR level of a word using the '*get_cefr ()*' method by passing the word itself and the part of speech used in the context of that sentence.
2. Under the 'SynRetriever' class file, we retrieve the relevant set of synonyms for a word. Here there are two helper methods to furnish the task
    a. 'retrieveSynsByPos ()' – Retrieves the synonyms list using the word itself and the PoS. This method is backed by the ThesaurusDict.json file.
    b. 'retrieveAllSyns ()'- Retrieves the synonyms list using the word alone without the necessity of the Part of Speech. This method is backed by both ThesaurusDict.json and SynonymDict.json files.

2.4.2 Apply Algorithmic Logic for Each Sentence

We iterate over the '*master_data*' list which has already been generated. Each sentence present in this list undergoes complexity reduction using the below stated workflow.



2.4.2.1 Complexity Reduction - Steps in our Algorithm:

1. Check the CEFR level of each word along with its corresponding PoS. Consider the word for complexity reduction, if the complexity of the word was either CEFR level B or C.
2. Shortlisted words proceed further to fetch their Synonyms List. Only words which receive a minimum list of size 1 are considered in the further steps of the algorithm.
3. Once the synonyms list has been stored for each word, we find the CEFR level of each word in the synonyms list. This step helps us in recognising the most eligible and easy to understand synonym word. The synonyms words are classified under 3 groups – CEFR level A, B & C.
4. Using the '***Roberta-Base*** *[2.5]*' Sentence Transformer technique we find the semantic relevance score of each word in our CEFR classified Synonyms List. Roberta Base has been pretrained on 5 datasets with the total weight of these datasets being 160GB. This has an STS benchmark of 85.44 which being the highest, considering the space and time trade off. Words with the highest relevance score from each of the CEFR level A, B & C list are shortlisted for further steps.
5. If the highest score among them, is not the most preferred reduction, we dictate another iteration using the synonyms list of the highest relevance score word.

CEFR Level	Most Preferred Reduction	Allowed Reduction
C	**A**	**C** [if the score is greater than the original], **B or A**
B	**A**	**B** [if the score is greater than the original] **or A**

[Reduction Reference Table]

6. The iteration mentioned in the previous step is limited to a maximum of 5 rounds. At the end of 5th round, we consider the results either from the most preferred reduction or from the set of allowed reductions.
7. Once the result word has been finalised, the candidate word in the sentence is replaced with the result word.
8. Process continues until the sentence is completely reduced in terms of complexity.

Key Stats:

Book Title	With GPU Acceleration (sec)	Without GPU Acceleration(sec)
Treasure Island	~2	~22
Inglorious Empire	~4	~50

*note that only 2 pages of the above books were used as a sample.

2.4.2.2 Checking Sentence Integrity

Now that the sentences are reduced to their most simplified form, we need to check their grammatical integrity to ensure that the sentences are not grammatically incorrect after replacing with our chosen synonym word. To achieve the same, we have used the python library known as GingerIt [4.1]. This library helps us in correcting spelling and grammar mistakes based on the context of complete sentences. It is a wrapper around the gingersoftware.com API.
After verifying and validating the sentence integrity we convert the list of modified sentences back into their paragraph form.

```
'''
Re pack sentences for pdf, based on the
pre-determined sentence count in a
paragraph
'''
for index,value in para_dictionary.items():
  '''
  This is a special case for PyMuPdf - Take care
  '''
  # if index>0 and index<len(para_dictionary):

  content = list_to_hold_sorted_data[:value]
  self.re_pack_sentence(index,content)
  # Mark \n and \t for the final json response
  marked_string.append(f"{''.join(content)}\n \t ")
```

2.5 GENERATE NEW PAGE

2.5.1 Reading the PDF in Python

In this section, the focus is on generating the new PDF after the processing of the page has finished. Here the challenge faced was to retain the paragraphs and the indentations of the original page after transforming the complex words. This was achieved with the help of "PyMuPDF" [2.4] and the "fitz" packages.

```
# Open the PDF Document as doc
self.doc = fitz.open(self.pdf_document)
# Load Page based on Page number
self.page = self.doc.loadPage(page_number)
# Split the page as paragraphs
self.page_block = self.page.getText("blocks")
```

The fitz module is given the path to the pdf document to load the document and then the required page numbers needed to convert it to a doc. Once the pdf is loaded the ***getText('blocks')*** function segregates the page into blocks which are stored in the form of an array in the class variable ***page_block.***

```
'''
Place \n and \t marker at the end of the paragraph
'''
marked_string = []
for i,v in enumerate(self.page_block):
    s = v[4]
    if i>0:
      marked_string.append(f"{s}\n \t")
    else:
      marked_string.append(f"{s}")
string = "".join(marked_string)
```

In the above code, we iterate over each page block and add a new line character(\n) and a tab(\t) in order to keep the indentation of the original text. These special characters are added as markers for the user interface.

2.5.2 Displaying the processed text on the user interface.

Once the user uploads the PDF book on the UI, it makes a REST call to the endpoint of the Flask application by first converting the file into a base64 processed string (byte stream) and then sending the string in the request along with the filename and the size of the file.

```
onFileChanged(event: any) {

  if (event && event.target && event.target.files.length > 0) {
    this.fileData = event.target.files[0];
    if (this.fileData) {
      const reader = new FileReader();
      reader.onload = this.handleReaderLoaded.bind(this);
      reader.readAsBinaryString(this.fileData);
    }
  }
}

handleReaderLoaded(e: any) {
  this.base64textString = btoa(e.target.result);
  const obj = {
    'bytes': JSON.stringify(this.base64textString),
    'filename': this.fileData.name,
    'size': this.fileData.size / 1000
  }
  this.makeCall(obj);
}
```

Angular code for PDF transformation

The server then processes the bytes and reads it as a file which is stored at a pre-decided location. It then reads the file from that location, processes the file for transformations and once complete, returns the output in the following format.

```
output = {
    "sentence_list": {…
    },
    "title": "Treasure Island",
    "formatted_text": `Off.What you mought call me? You mought call me captain.
                Oh, I see what you're at\u2014there\"; and he threw down three or four gold pieces on the thr
                \"You can tell me when I've worked through that,\" says he, looking as angry as a commander.
                \n \t And very bad as his clothes were and coarsely as he spoke, he had none of the plan of
                \n \t The man who came with the barrow told us the mail had set him down the morning before
}
```

Response given back to the user interface.

To display the text on the user interface, only the value of the variable ***"formatted_text"*** is taken and processed. The text is converted into HTML format in order to retain the paragraphs and indentation where every **"\n"** is converted to a **"
"** tag and every **"\t"** is converted to **"   "** to replicate 4 spaces.

2.5.3 Highlighting the transformed words on the User Interface.

To process the transformed words, the "sentence_list" variable of the response is used.

```
"sentence_4": {
    "index_1": {
        "word": "very",
        "score": 0.9984726905822754,
        "cefr": "A",
        "sentence": "And very bad as his clothes were and coarsely as he spoke, he had none of the appe
        "children": [{
            "word": "indeed",
            "cefr": "C",
            "sentence": " And indeed bad as his clothes were and coarsely as he spoke, he had none of t
        }]
    },
    "index_17": {…
    },
    "index_30": {
        "word": "classmate",
        "score": 0.9683722257614136,
        "cefr": "A",
        "sentence": "And very bad as his clothes were and coarsely as he spoke, he had none of the plan
        "children": [{
            "word": "mate",
            "cefr": "B",
            "sentence": "And very bad as his clothes were and coarsely as he spoke, he had none of the
        }]
    }
},
```


"sentence_list" sub-structure

The variables and their use are mentioned below.

Variable	**Use**
sentence_4	The index of the sentence to be operated upon.
Index_1/index_30	The index of the word in the sentence that has undergone transformation.
word	Transformed word.
score	The final relevance score of the word in the sentence
cefr	CEFR level of the transformed word
sentence	Sentence with the original word replaced by the transformed word.
children	This is the upper structure which repeats showing us the order of the transformation and used to show the hierarchy tree on the user interface.

From the exact index of the word and the sentence, we were able to highlight the transformed word by adding a **"<mark>"** tag before and after the word. This step is repeated for all the words in a sentence and then for all the sentences in the page. Once complete the transformed text is displayed on the user interface.

3. PROGRAMMING CONCEPT HIGHLIGHT

1. Usage of **Priority Queue** Data structure in order to reduce the time complexity to O(1) for fetching the semantically relevant and highest-ranking word.
2. Usage of **Multi-Thread** concept to reduce the overload on a single thread. This concept is utilised during the grammar check for each sentence. Since the grammar API was time consuming, we were dictated to use threads to fasten up the process.
3. Working along with Pandas & NumPy.
4. Working along with GPU and CPU based performances.
5. Usage of Flask® to render the front-end files.
6. Usage of ngrok® to re-direct local host requests to a remote server.
7. Understandings about a PDF file and its co-ordinate system for writing each line.
8. Using Angular/HTML/CSS for the front-end.
9. Packages and the methods used for web-scraping.

4. PROSPECTS

The current limitation of our project is the database. The current size of our word CEFR database comprises of 7621 words which for a real-scale project is not enough.

As an enhancement to our project, we plan to train a model which can be used to detect or identify possible complex words. Once the model is appropriately trained, we would not require the use of a database anymore to judge the CEFR level of a word. With this approach either we would also have to follow the dynamic web-scraping approach mentioned earlier or we would have to look at other alternatives. According to the research conducted by Armand Olivares [3.1], BERT can be further used to provide a simpler substitution for a complex word.

Once we've integrated Machine Learning into this project, we also want to develop a full-fledged feature rich e-reader to provide the experience to users on their reading devices replacing the browser output currently present.

5. CONCLUSION

To conclude the report on a high note, we were successful in solving the problem statement which we aimed at solving right from the beginning which was "***Reducing the complexity of a sentence by replacing complex words with much simpler words that can easily be understood***". By achieving this target, we have laid a foundation to a prospect which aims at being more flexible, reliable, and effective. Some key features of our projects consist of:

1. Usage of CUDA acceleration to improvise model training and semantic analysis.
2. Intuitive UI designed using NodeJS, Angular and Flask.
3. Algorithmic efficiency is clocked no more than $O(n^2)$.
4. Design comprising of Front-End Logic, Web Scraping & Backend Python framework
5. Understanding of Word to Vector logic.

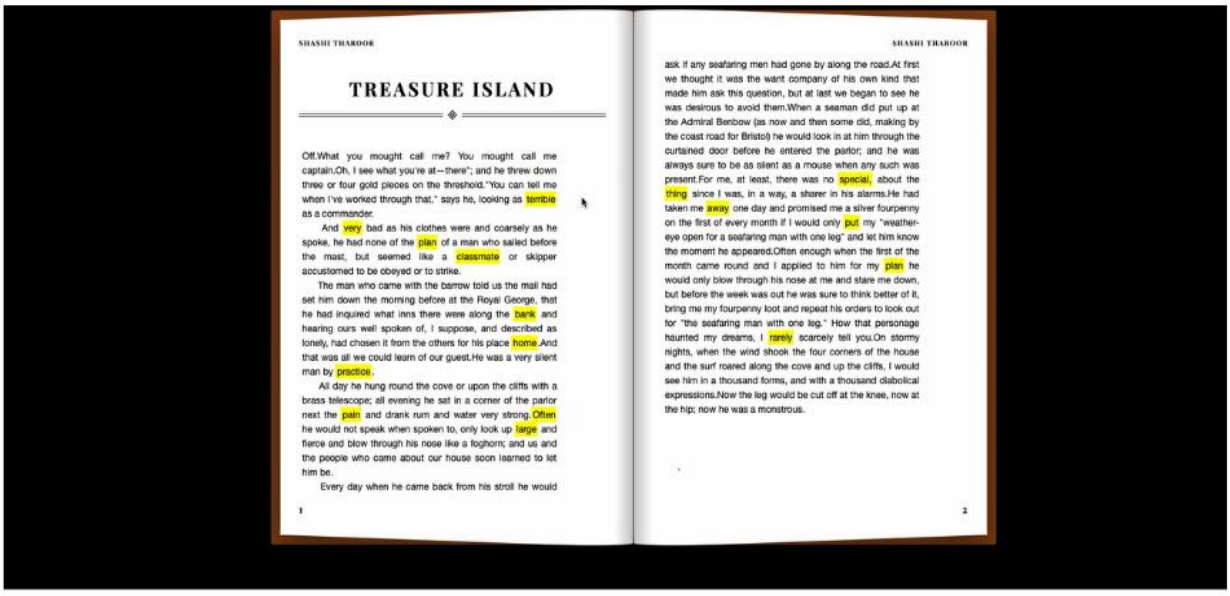
However, there are some limitations in our current implementation. Few significant ones are,

1. Runtime complexity needs to be improved – Current Runtime with GPU acceleration for a Single Page PDF is roughly 14-20 secs.
2. Grammar Checking needs to be improved. Python library GingerIt, [4.1] consumes nearly 10-12 secs per page.
3. Erroneous or Invalid Synonym substitution on rare occasions.
4. Inability to used trained models to improve run time.

Output Series

[Output 1 - Introduction Page – Created Using NodeJS, Flask and Ngrok]

[Output 2 – Backend Python Logic in execution]


SHASHI THAROOR

TREASURE ISLAND

Off.What you mought call me? You mought call me captain.Oh, I see what you're at—there"; and he threw down three or four gold pieces on the threshold."You can tell me when I've worked through that," says he, looking as terrible as a commander.

And very bad as his clothes were and coarsely as he spoke, he had none of the plan of a man who sailed before the mast, but seemed like a classmate or skipper accustomed to be obeyed or to strike.

The man who came with the barrow told us the mail had set him down the morning before at the Royal George, that he had inquired what inns there were along the bank and hearing ours well spoken of, I suppose, and described as lonely, had chosen it from the others for his place home.And that was all we could learn of our guest.He was a very silent man by practice.

All day he hung round the cove or upon the cliffs with a brass telescope; all evening he sat in a corner of the parlor next the pain and drank rum and water very strong.Often he would not speak when spoken to, only look up large and fierce and blow through his nose like a foghorn; and us and the people who came about our house soon learned to let him be.

Every day when he came back from his stroll he would

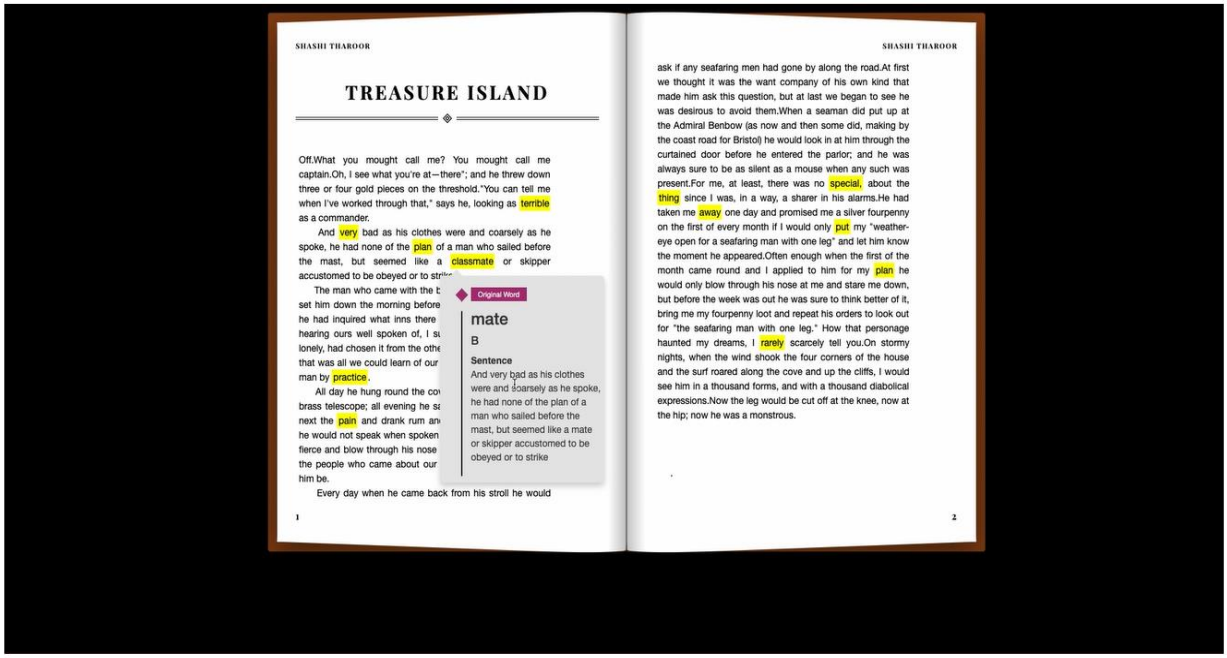
1

SHASHI THAROOR

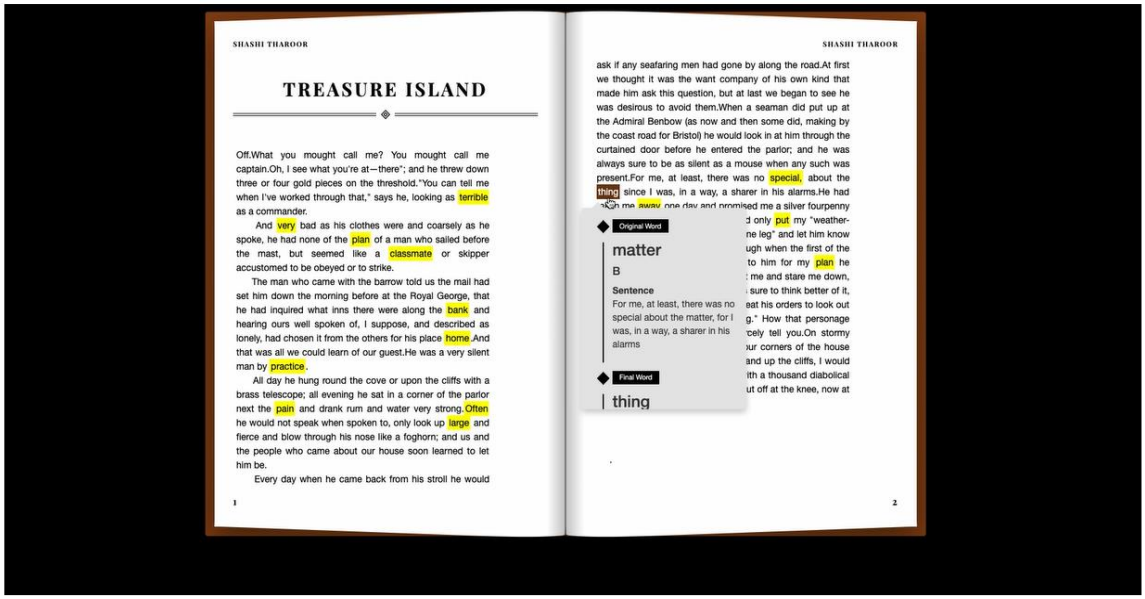
ask if any seafaring men had gone by along the road.At first we thought it was the want company of his own kind that made him ask this question, but at last we began to see he was desirous to avoid them.When a seaman did put up at the Admiral Benbow (as now and then some did, making by the coast road for Bristol) he would look in at him through the curtained door before he entered the parlor; and he was always sure to be as silent as a mouse when any such was present.For me, at least, there was no special, about the thing since I was, in a way, a sharer in his alarms.He had taken me away one day and promised me a silver fourpenny on the first of every month if I would only put my "weather-eye open for a seafaring man with one leg" and let him know the moment he appeared.Often enough when the first of the month came round and I applied to him for my plan he would only blow through his nose at me and stare me down, but before the week was out he was sure to think better of it, bring me my fourpenny loot and repeat his orders to look out for "the seafaring man with one leg." How that personage haunted my dreams, I rarely scarcely tell you.On stormy nights, when the wind shook the four corners of the house and the surf roared along the cove and up the cliffs, I would see him in a thousand forms, and with a thousand diabolical expressions.Now the leg would be cut off at the knee, now at the hip; now he was a monstrous.

2

[Output 3 – Output Generated upon computation – Highlighted words are the words which are already replaced. Hovering over them would provide more information regarfing the same]



[Output 4 – Hovering over the highlighted words gives more information regarding the list of eligible candidates for the word along with their sematic score]



[Output 5- Information about a different word]

References

[2.1] Cambridge Assessment English. (n.d.). International language standards | Cambridge English. https://www.cambridgeenglish.org/exams-and-tests/cefr/#:%7E:text=The%20Common%20European%20Framework%20of%20Reference%20for%20Languages%20(CEFR)%20is,who%20have%20mastered%20a%20language.

[2.3] English Profile - EVP Online. (n.d.). English Profile. https://www.englishprofile.org/wordlists/evp

[2.3.1] BeautifulSoup4 documentation https://pypi.org/project/beautifulsoup4/

[2.3.2] Requests Documentation https://pypi.org/project/requests/

[2.4] PyMuPDF Documentation — PyMuPDF 1.18.9 documentation. (n.d.). PyMuPDF. https://pymupdf.readthedocs.io/en/latest/index.html

[2.5] roberta-base · Hugging Face. (n.d.). Roberta Base. https://huggingface.co/roberta-base

[3.1] Lexical Simplification using BERT https://medium.com/@armandj.olivares/how-to-use-bert-for-lexical-simplification-6edbf5a4d15e

[4.1] gingerit. (2021, February 1). PyPI. https://pypi.org/project/gingerit/